



Paradigme de proiectare a aplicatiilor web

Curs 2025-2026

MODEL VIEW CONTROLLER

Tehnologii utilizate in dezvoltarea aplicatiilor web

- ▶ Baza de date - SQL Server
 - ▶ Notiuni: baza de date, schema, tabel, vedere, functie, procedura stocate, trigger
 - ▶ Relatii
 - ▶ Utilizatori – Security -> Logins
 - ▶ Backup (logic si fizic), generare scripturi update
- ▶ Mediu de dezvoltare
 - ▶ .NET Framework 4.8 (Last version) – doar pentru Windows [1]
 - ▶ .NET Core – cross platform [2]
- ▶ Server Web – IIS (Windows Components)

Tipuri de arhitecturi aplicatii web

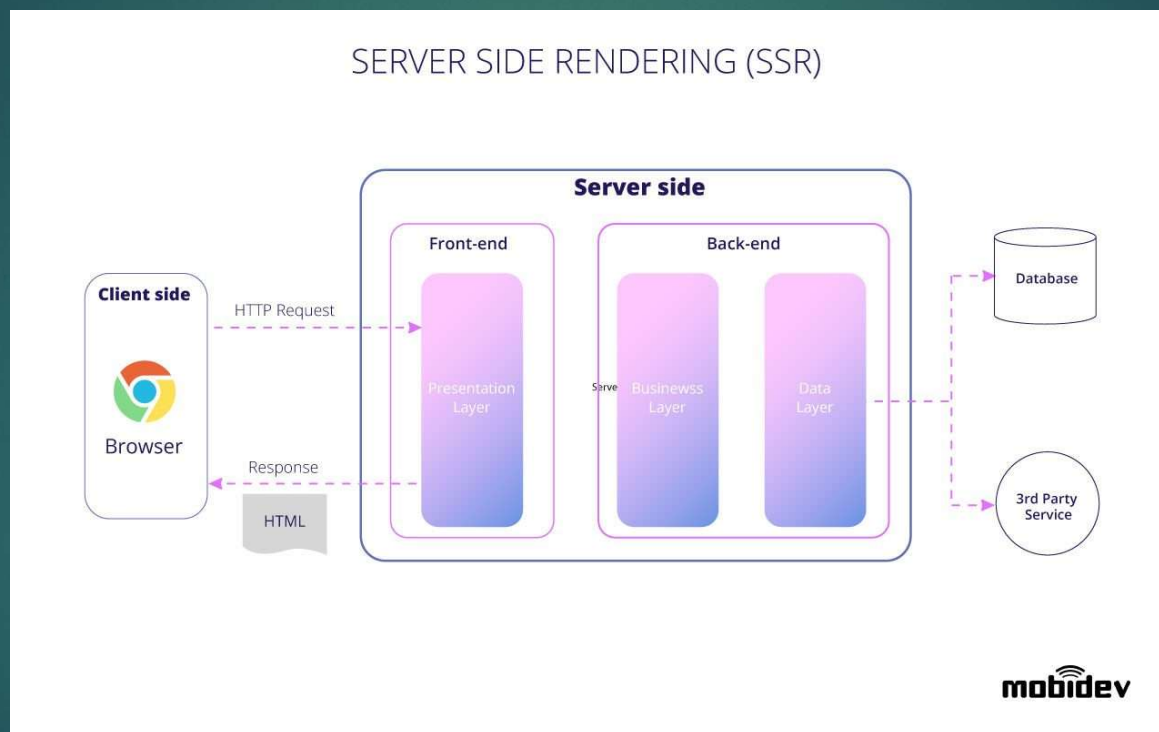


Figura 1. Arhitectura server side rendering [3]

Tipuri de arhitecturi aplicatii web

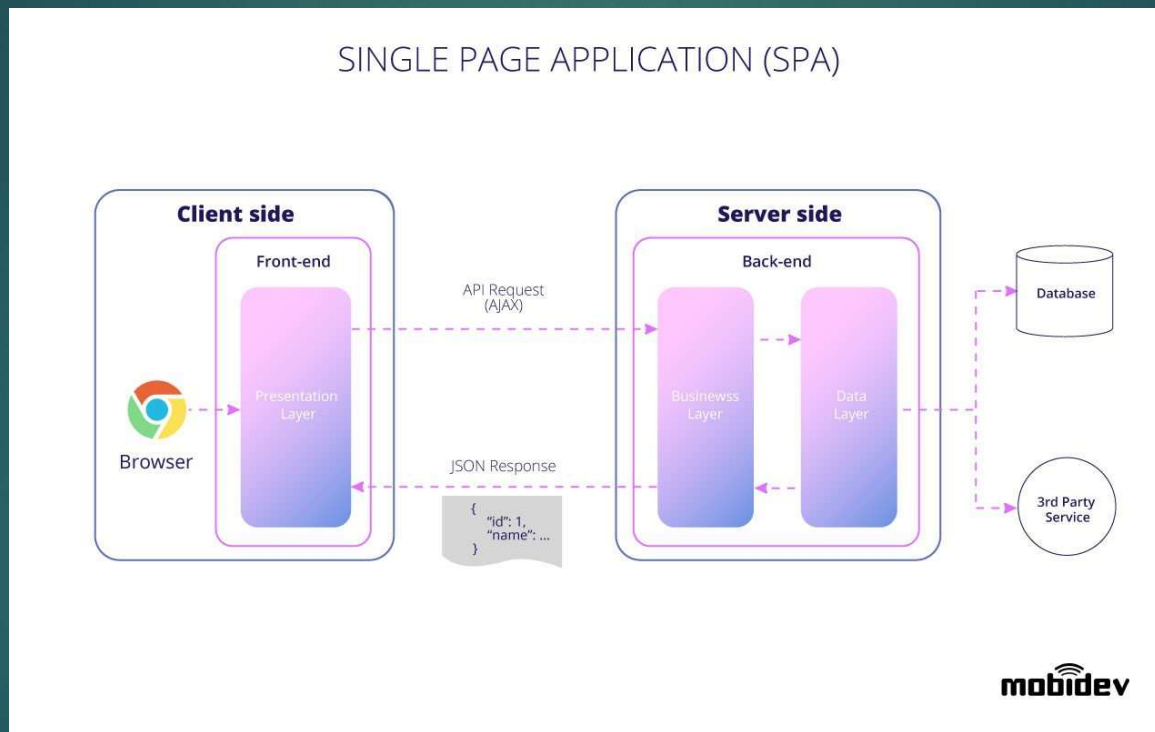


Figura 2. Arhitectura SPA (SINGLE PAGE APPLICATION)[3]

Paradigma MVC

- ▶ Model – clase ce reprezinta domeniul aplicatiei
- ▶ View – template pentru a genera pagina HTML
- ▶ Controller – gestioneaza relatiile dintre View si Model
- ▶ Routes

De retinut:

- ▶ Folosita in arhitectura server side rendering
- ▶ *Un URL nu este egal cu o pagina.*
 - ▶ In ASP.NET Web Forms este o corespondenta intre URL si pagina (pentru fiecare pagina se apeleaza fisierul *.aspx corespunzator).
 - ▶ In Asp MVC cererile din browser sunt mapate la actiuni din controller prin intermediul rutelor (**Routes**). S-a eliminat dependenta directa dintre url si paginile accesate.

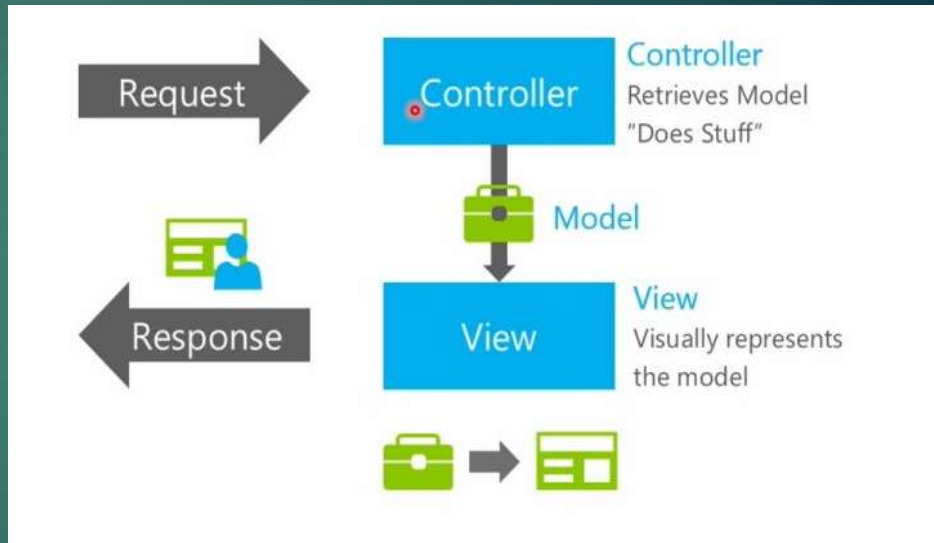


Figura 1: Paradigma MVC [1]

Definirea rutelor

- ▶ Se folosește o tabelă de rutare pentru a trata cererile care apar.
- ▶ Metoda folosită pentru a înregistra o *ruta* este **RegisterRoutes** în care se adaugă o intrare în tabelul cu rute. Codul se plasează în *Global.asax* în cadrul apelului metodei **Application_Start**.
- ▶ O ruta este de forma:
 - ▶ "{controller}/{action}/{id}"
- ▶ O ruta implicită este definită sub forma:
`new { controller = "Home", action = "Index", id = "" }`

```
0 references
protected void Application_Start()
{
    AreaRegistration.RegisterAllAreas();
    FilterConfig.RegisterGlobalFilters(GlobalFilters.Filters);
    RouteConfig.RegisterRoutes(RouteTable.Routes);
    BundleConfig.RegisterBundles(BundleTable.Bundles);
}
```

```
1 reference
public static void RegisterRoutes(RouteCollection routes)
{
    routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

    routes.MapRoute(
        name: "Default",
        url: "{controller}/{action}/{id}",
        defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
    );
}
```

Custom routes

- ▶ Rute cu elemente statice
- ▶ Ordinea de evaluarea a rutelor

```
public static void RegisterRoutes(RouteCollection routes)
{
    routes.MapRoute(
        name: "AdaugareMasinaAdmin",
        url: "Admin/{controller}/Adauga",
        new { controller = "Masina", action = "Create" }
    );

    routes.MapRoute(
        name: "Default",
        url: "{controller}/{action}/{id}",
        defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
    );
}
```

Controler

- ▶ Controler-ul este responsabil cu prelucrarea datelor venite de la utilizator (raspunde la interactiunile utilizatorului cu aplicatia).
- ▶ Un controler este o clasa al carei nume trebuie sa se termine cu *Controller* si este derivat din clasa **Controller**.
- ▶ Restrictii metode Action:
 - ▶ Trebuie sa fie publice. Nu pot fi private sau protected.
 - ▶ Nu pot fi statice.
- ▶ Tipuri returnate din metodele Action [2]:
 - ▶ Un view personalizat cu datele dintr-un model – exp. `ViewResult`
 - ▶ Obiecte in format Json sau XML – exp. `JsonResult`
 - ▶ Un text – exp. `ContentResult`
 - ▶ Redirect – exp. `Redirect`

```
public class CompaniesController : Controller
{
    IstocareCompanii stocareCompanii = (IstocareCompanii)new StocareFactory().GetTipStocare(typeof(Companie));

    // GET: Companies
    public ActionResult Index()
    {
        return View(stocareCompanii.GetCompanii());
    }

    // GET: Companies/Create
    public ActionResult Create()
    {
        return View(new Companie());
    }

    // POST: Companies/Create
    [HttpPost]
    public ActionResult Create(Companie model)
    {
        try
        {
            // TODO: Add insert logic here
            stocareCompanii.AddCompanie(model);
            return RedirectToAction("Index");
        }
        catch
        {
            return View();
        }
    }
}
```

Model

- ▶ Clasa care reprezinta **Modelul** in paradigma MVC cuprinde datele specifice domeniului.
 - ▶ Proprietatile descriu entitatile
 - ▶ Metodele descriu business logic
- ▶ Intr-o aplicatie MVC clasele Model ar trebui create in directorul special **Model**.
- ▶ Modelul este utilizat in view pentru a popula controalele de pe pagina cu date din model si, de asemenea, pentru a transmite date catre controller.

```
public class Companie
{
    19 references
    public int IdCompanie { get; set; }
    19 references
    public string Nume { get; set; }
    15 references
    public string Email { get; set; }
    15 references
    public long Telefon { get; set; }
    15 references
    public string Adresa { get; set; }

    1 reference
    public Companie()
    { }
}
```

Custom Model

Modelele custom sunt necesare in urmatoarele cazuri:

- ▶ Date suplimentare pentru UI – Exemplu: elementele pentru liste de selectie (dropdowns)
- ▶ Entitati complexe – care au nevoie de stocare in mai multe tabele (exemplu: factura).

Observatie: Se recomanda utilizarea modelelor custom pentru UI chiar daca nu sunt schimbari fata de modelul din BD.

```
public class CreateMasinaModel
{
    public Masina Masina { get; set; }
    public List<Companie> Companii { get; set; }

    public CreateMasinaModel()
    {
        Companii = new List<Companie>();
        Masina = new Masina();
    }
    public CreateMasinaModel(List<Companie> companii)
    {
        Masina = new Masina();
        this.Companii = companii;
    }
}
```

View

- ▶ View este echivalent cu pagina web care va fi randata la client.
- ▶ Regula generala:
 - ▶ Pentru fiecare actiune exista un view cu acelasi nume si extensia `.cshtml`, fisier plasat in directorul:
`\Views\<nume_controller_fara_sufix_Controller>\`
 - ▶ Exemplu de view-uri pentru actiunile `Index()` si `Create()`:
 - ▶ **`\Views\Companies\Index.cshtml`**
 - ▶ **`\Views\Companies\Create.cshtml`**
- ▶ Razor View Engine
 - ▶ Compileaza un view cu un amestec de tag-uri HTML si cod server-side
 - ▶ Foloseste caracterul `@` pentru a include codul server-side

```
@model IEnumerable<LibrarieModele.Companie>

@{
    ViewBag.Title = "Index";
}

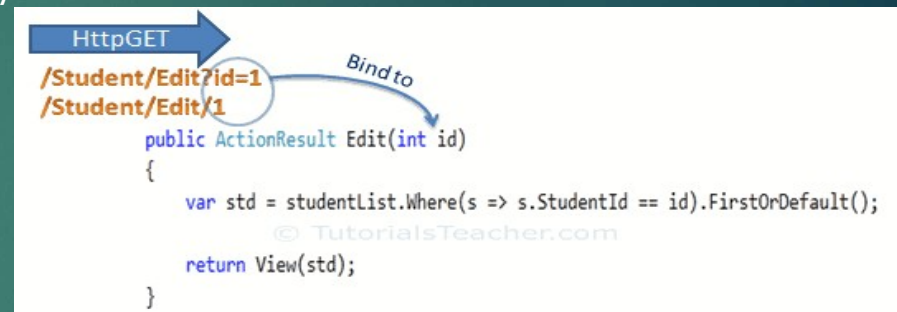
<h2>Index</h2>

<p>
    @Html.ActionLink("Create New", "Create")
</p>
<table class="table">
    <tr>
        <th>
            @Html.DisplayNameFor(model => model.IdCompanie)
        </th>
        <th>
            @Html.DisplayNameFor(model => model.Nume)
        </th>
        <th>
            @Html.DisplayNameFor(model => model.Email)
        </th>
        <th>
            @Html.DisplayNameFor(model => model.Telefon)
        </th>
        <th>
            @Html.DisplayNameFor(model => model.Adresa)
        </th>
    </tr>
</table>

@Foreach (var item in Model) {
    <tr>
        <td>
            @Html.DisplayFor(modelItem => item.IdCompanie)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.Nume)
        </td>
    </tr>
}
```


Model Binding

- ▶ Corespondenta se realizeaza pe baza de nume (nume de variabila sau nume ale proprietatilor unui obiect)
- ▶ Binding catre tipuri elementare :



- ▶ Binding catre tipuri complexe:



Procesul de Model Binding

- ▶ Model binding este un proces in 2 etape:
 - ▶ Colectarea valorilor din requestul HTTP
 - ▶ Popularea datelor primitive sau complexe cu aceste valori.
- ▶ In etapa de colectare a valorilor se evalueaza datele din urmatoarele surse:
 - ▶ Campuri din formular (Request.Form)
 - ▶ Parametri din Query string (Request.QueryString)
 - ▶ Date din ruta (RouteData.Values)
 - ▶ JSON Request body (Request.InputStream) cand este un AJAX request
 - ▶ Fișiere primite prin POST (Request.Files)

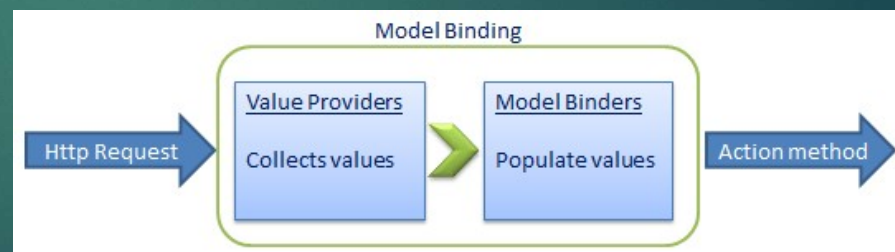


Figura 2. Model Binding in ASP.NET MVC

Bibliografie

1. https://en.wikipedia.org/wiki/.NET_Framework_version_history
2. https://en.wikipedia.org/wiki/.NET_Core
3. <https://mobidev.biz/blog/web-application-architecture-types>
4. <https://medium.com/@theaxontech/asp-net-mvc-overview-a2ac7660539f>
5. <https://www.tutorialsteacher.com/mvc/action-method-in-mvc>
6. <https://www.tutorialsteacher.com/mvc/model-binding-in-asp.net-mvc>
7. <https://docs.microsoft.com/en-us/aspnet/mvc/overview/getting-started/introduction/getting-started>
8. <https://docs.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-3.1>
9. <https://www.tutorialsteacher.com/mvc/create-edit-view-in-asp.net-mvc>